

Secure Cloud Document Portal — Final Report

Cryptography Coursework

May 8, 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
1.3	Scope	3
2	Background	3
2.1	Symmetric Encryption — Blowfish	3
2.2	Stream Ciphers — ChaCha20	3
2.3	Public Key Cryptography — ElGamal	4
2.4	Certificate Authorities	4
3	System Design	4
3.1	Architecture	4
3.2	Data Flow	4
3.3	Threat Model	4
4	Implementation	5
4.1	Blowfish	5
4.2	ChaCha20	5
4.3	ElGamal	5
4.4	Certificate Authority	5
4.5	Authentication	5
4.6	Client-Server Communication	5
5	Testing	5
5.1	Unit Tests	5
5.2	Integration Test	6
5.3	Results	6
6	Performance Analysis	6
6.1	Encryption Speed	6
6.2	Ciphertext Expansion	6
6.3	Key Generation	6
6.4	Observations	6

7	Conclusion	6
7.1	Future Work	7
8	References	7
9	Project Reflection and Self-Evaluation	8
9.1	What I Used in This Project	8
9.2	What Was the Most Difficult Part	8
9.3	What I Learned	8
9.4	Limitations of the Project	9
9.5	Self-Evaluation / Rating	9
9.6	Future Improvements	10

1 Introduction

1.1 Problem Statement

Cloud storage systems require multiple layers of cryptographic protection to ensure confidentiality, integrity, and authenticity of user data. This project implements a secure cloud document portal with encryption at rest, encryption in transit, authentication, key management, and a simulated Certificate Authority.

1.2 Objectives

- Implement Blowfish, ChaCha20, and ElGamal algorithms from scratch
- Build a client-server document portal with dual encryption layers
- Simulate a PKI with a Certificate Authority
- Demonstrate secure key exchange and session management
- Provide comprehensive testing and performance analysis

1.3 Scope

The system handles document upload/download with encryption, user authentication, certificate-based trust, and key rotation. It uses Python standard libraries only — no external cryptographic packages.

2 Background

2.1 Symmetric Encryption — Blowfish

Blowfish is a symmetric block cipher designed by Bruce Schneier in 1993. It uses a 16-round Feistel network with 64-bit blocks and supports keys from 32 to 448 bits. The algorithm features key-dependent S-boxes, initialized from the hex digits of π and then modified through the key schedule process.

Key concepts: Feistel network, S-boxes, P-array, CBC mode, PKCS7 padding.

2.2 Stream Ciphers — ChaCha20

ChaCha20, designed by Daniel Bernstein, is a stream cipher using only Add-Rotate-XOR (ARX) operations. It takes a 256-bit key and 96-bit nonce, builds a 4×4 state matrix, and applies 20 rounds of quarter-round mixing to generate keystream blocks.

Key concepts: ARX operations, quarter round, keystream generation, XOR encryption.

2.3 Public Key Cryptography — ElGamal

The ElGamal cryptosystem, based on the Diffie-Hellman key exchange, provides both encryption and digital signatures. Security relies on the difficulty of the Discrete Logarithm Problem in finite fields.

Key concepts: Discrete logarithm, safe primes, generators, modular exponentiation.

2.4 Certificate Authorities

A CA is a trusted third party in a Public Key Infrastructure (PKI) that issues digital certificates binding public keys to identities. Our simulation implements certificate issuance, signature verification, and revocation.

3 System Design

3.1 Architecture

The system consists of a Client, Server, and CA.

- **Client:** Handles upload/download, connects to server, uses ChaCha20.
- **Server:** Handles authentication, storage with Blowfish, key rotation.
- **CA:** Issues and verifies ElGamal signatures for certificates.

3.2 Data Flow

1. CA generates root keypair and issues certificates
2. Client connects to server via TCP socket
3. ElGamal key exchange establishes shared session key
4. All subsequent communication encrypted with ChaCha20
5. Uploaded files encrypted with Blowfish CBC for storage
6. Downloads reverse the encryption layers

3.3 Threat Model

Threat	Mitigation
Eavesdropping on network	ChaCha20 encrypted channel
Server disk compromise	Blowfish CBC file encryption
Password theft	Salted SHA-256 hashing
Man-in-the-middle	Certificate-based authentication
Key compromise	Key rotation capability
Replay attacks	Random session keys + nonces

4 Implementation

4.1 Blowfish

The Blowfish implementation uses a custom Key Schedule and Feistel rounds. The F function splits a 32-bit input into four 8-bit pieces, processing them through four key-dependent S-boxes. Files are padded with PKCS7 and encrypted in CBC mode using a random IV.

4.2 ChaCha20

ChaCha20 implementation strictly adheres to the 20-round ARX sequence (column + diagonal) to generate blocks of keystream for XOR operations. The state matrix consists of constants, key, counter, and nonce.

4.3 ElGamal

ElGamal uses the Discrete Logarithm Problem to perform key exchange. We implemented custom modular exponentiation and the Extended Euclidean Algorithm for modular inverses (required for signature generation and verification).

4.4 Certificate Authority

The CA signs the public key and identity of users/servers using its ElGamal private key. The resulting certificate is passed to clients, who verify it using the CA's well-known root public key.

4.5 Authentication

Passwords are hashed using SHA-256 along with a randomly generated 16-byte salt per user.

4.6 Client-Server Communication

A custom binary framing protocol is used over standard TCP sockets to allow JSON payload lengths to be parsed correctly before the ChaCha20 stream decryption.

5 Testing

5.1 Unit Tests

- **Blowfish:** 7 tests (ECB, CBC, IVs, keys, padding, files, large data)
- **ChaCha20:** 8 tests (encrypt, symmetry, nonces, keys, expansion, multi-block, RFC vector)
- **ElGamal:** 8 tests (keygen, encrypt, sign, tamper, key exchange, session key, bytes, serialization)
- **Auth:** 10 tests (register, duplicate, login, wrong password, sessions, hashing)

- **CA:** 6 tests (issue, verify, tamper, revoke, multiple, root key)

5.2 Integration Test

Full system flow: CA setup → certificate issuance → client connection → handshake → registration → login → file upload → listing → download → integrity check → key rotation → re-download.

5.3 Results

All 40+ tests pass successfully.

6 Performance Analysis

6.1 Encryption Speed

Size	Blowfish CBC	ChaCha20	Winner
1 KB	0.97 ms	1.29 ms	Blowfish
10 KB	9.37 ms	12.25 ms	Blowfish
100 KB	124.73 ms	122.00 ms	ChaCha20

6.2 Ciphertext Expansion

Blowfish (block cipher) has +4 to +8 bytes of padding overhead. ChaCha20 (stream cipher) has zero expansion overhead.

6.3 Key Generation

For ElGamal, 256-bit prime generation takes ~1.6 seconds, while 512-bit prime generation takes ~74 seconds in pure Python.

6.4 Observations

ChaCha20 is faster for larger data (stream cipher advantage). Blowfish has padding overhead; ChaCha20 has zero expansion. ElGamal keygen is the bottleneck due to safe prime generation.

7 Conclusion

The project successfully implements a multi-layered cryptographic system using only Python standard libraries. All three algorithms (Blowfish, ChaCha20, ElGamal) work correctly as verified by comprehensive testing. The system demonstrates encryption at rest, encryption in transit, secure authentication, key management, and certificate authority simulation.

7.1 Future Work

- Implement AES for comparison
- Add file integrity checking (HMAC)
- Support concurrent multi-user access
- Implement certificate chain of trust
- Add TLS-like protocol negotiation

8 References

1. Schneier, B. "Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish)." 1993.
2. Bernstein, D.J. "ChaCha, a variant of Salsa20." 2008.
3. ElGamal, T. "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms." 1985.
4. RFC 7539: "ChaCha20 and Poly1305 for IETF Protocols."
5. Menezes, A., van Oorschot, P., Vanstone, S. "Handbook of Applied Cryptography." CRC Press.

9 Project Reflection and Self-Evaluation

9.1 What I Used in This Project

I built this project entirely in Python using only standard libraries like `socket`, `json`, `hashlib`, `random`, `time`, and `os`. My goal was to strictly avoid external cryptographic packages (like `cryptography` or `PyCryptodome`) to really understand how the math and algorithms work under the hood.

For the cryptography side, I implemented three main algorithms from scratch:

- **Blowfish (CBC mode):** Used for encrypting files stored on the server (encryption at rest). I chose this because building a Feistel network is a classic learning exercise, and its 64-bit blocks with variable-length keys felt like a good fit for file storage.
- **ChaCha20:** Used to encrypt the actual socket communication (encryption in transit). I wanted a stream cipher here because it doesn't require block padding and is extremely fast, making the client-server chat feel responsive without lag.
- **ElGamal:** Handled key exchange and digital signatures. The Diffie-Hellman style key exchange was perfect for establishing the ChaCha20 session keys without ever sending them over the network.

The system is split into three main components: a Client, a Server, and a simulated Certificate Authority (CA). The CA was crucial because I needed a way to prove that the server the client connects to is actually the real server (preventing man-in-the-middle attacks). Every single cryptographic function, from the Miller-Rabin primality tests to the ChaCha quarter rounds, was written completely from scratch.

9.2 What Was the Most Difficult Part

Without a doubt, the ElGamal implementation and the underlying number theory was the toughest part.

Getting the modular exponentiation and prime generation to work efficiently was a headache. At first, my prime generation was taking way too long, or generating numbers that weren't "safe primes," which meant the generator g wouldn't cover the full mathematical group properly. I had to go back and really study the math behind finding primitive roots and implementing the Extended Euclidean Algorithm for modular inverses (which is needed for the ElGamal signature generation and decryption).

Debugging this was tricky because a math error just results in gibberish decryption or failed signatures, with no clear indication of *which* step went wrong. I solved this by writing a small-number test script (`demo_small_numbers.py`) using tiny primes like 23 so I could trace the calculations by hand on paper. Once the small numbers worked, scaling it up to 256 or 512 bits worked perfectly.

9.3 What I Learned

This project was a massive learning experience. Specifically:

- **Symmetric vs. Asymmetric:** I finally grasped *why* we use both in modern systems. Asymmetric (ElGamal) is mathematically elegant but incredibly slow.

Using it just to securely exchange a symmetric key (ChaCha20) which then handles the heavy lifting is a concept that makes complete practical sense to me now.

- **Key Management is Hard:** Writing the encryption algorithm is only half the battle. Storing the keys, rotating them, deriving session keys, and making sure the client and server agree on them without leaking them was much more complex than the actual XOR operations.
- **Protocol Design:** I learned how to structure a basic secure protocol. Doing the CA handshake first, verifying the signature, then doing the DH key exchange, and *then* starting the encrypted channel mirrors how real TLS works, just on a simpler scale.
- **Performance Tradeoffs:** Using 256/512-bit primes instead of 2048-bit for ElGamal was a deliberate tradeoff to keep the demo running smoothly. It highlighted that security often directly competes with performance and user experience.

9.4 Limitations of the Project

While it works well for a coursework project, it's definitely not production-ready:

- **Custom Crypto is Risky:** Implementing crypto from scratch is famously dangerous. My code hasn't been audited for side-channel attacks (like timing attacks during modular exponentiation).
- **Key Sizes:** I defaulted to smaller primes for ElGamal to make the demo run faster. Real-world systems use 2048+ bits, but calculating that in pure Python takes too long.
- **Replay Attacks:** While I used nonces in ChaCha20, my socket protocol doesn't have a strict sequence numbering system to prevent sophisticated replay or message reordering attacks over the network.
- **Basic CA:** The CA is simulated and lives locally. In a real system, certificate validation involves checking complex trust chains (X.509) and online revocation lists (OCSP).

9.5 Self-Evaluation / Rating

Technical implementation: 9/10

I successfully coded three complex algorithms (Blowfish, ChaCha20, ElGamal) entirely from scratch without libraries. They work correctly across file boundaries and live socket streams.

Security design: 8/10

The architecture is solid (defense in depth with at-rest and in-transit encryption, salted passwords, CA validation). It loses a point only because of the inherent risks of custom implementations and the lack of robust network replay protection.

Code quality: 8/10

The code is modular, separated by concern (auth, crypto, ca, server, client), and heavily commented. I included testing suites which helps, but some of the raw TCP socket handling could be more robust against unexpected network drops.

System integration: 9/10

The way the CA, client, and server interact is seamless. The automatic derivation of ChaCha20 session keys from the ElGamal shared secret worked better than I initially expected.

Overall completeness: 9/10

It meets all the coursework requirements, includes full unit/integration testing, performance benchmarks, and a working demo that simulates the entire lifecycle of a user interacting with the portal.

Overall final score: 8.6/10

I'm very proud of this project. It goes beyond just writing a cipher by building a complete ecosystem around it. It proved to be a challenging but highly rewarding exercise in applied cryptography.

9.6 Future Improvements

If I had more time or were to take this further, I would focus on:

- **HMAC for Integrity:** Adding a Message Authentication Code (like HMAC-SHA256) to the ChaCha20 stream to ensure the ciphertext hasn't been tampered with (Authenticated Encryption).
- **Better Socket Protocol:** Upgrading the raw TCP sockets to handle dropped packets gracefully and prevent message reordering.
- **Graphical Interface:** Building a simple Tkinter or PyQt UI so users don't have to use the terminal.
- **Post-Quantum Considerations:** It would be interesting to research and perhaps implement a toy version of a lattice-based key exchange algorithm, knowing that algorithms like ElGamal are theoretically vulnerable to Shor's algorithm on quantum computers.